

provide the table contain the types of IPC and used API with param usage details in table and file are were writre(user or kernel sopace) and path specification for file(/ must for file start of message queue and SHM) other required infor for interview point of view with contain the flow diagram Note: To give the separate table with Communication, Visible Locatio

That's a great idea. For interview preparation, it's much more useful to have a **single reference sheet** comparing all IPC mechanisms.

Below is a consolidated reference that covers:

- APIs
- Parameters
- Kernel/User space
- Visible locations
- Naming rules
- Communication flow
- Interview points

IPC Mechanism Comparison

IPC	Communication	Direction	Copy Data	Sync Required	Kernel Buffer	Best For
Pipe	Parent ↔ Child	One-way	Yes	No	Yes	Simple parent-child

IPC	Communication	Direction	Copy Data	Sync Required	Kernel Buffer	Best For
FIFO (Named Pipe)	Unrelated Processes	One-way	Yes	No	Yes	Local IPC
POSIX Message Queue	Unrelated Processes	Two-way (using two queues)	Yes	No	Yes	Events, Comm
Shared Memory	Any Processes	Two-way	No	Yes (Semaphore/Mutex)	No	Large Data
Semaphore	Synchronization	N/A	No	Itself	Kernel	Protect Shared Memory
Mutex (Process Shared)	Synchronization	N/A	No	Itself	Shared Memory	Critical Section
UNIX Domain Socket	Local Processes	Full Duplex	Yes	No	Yes	WebKit, Chrom
TCP Socket	Network	Full Duplex	Yes	No	Yes	Client-Server
mmap(File)	Shared File	Two-way	No	Yes	No	Large Files
Signals	Notification	One-way	No	No	No	Events

Important API Comparison

IPC	Create/Open	Send	Receive	Close	Delete
Pipe	pipe()	write()	read()	close()	Auto
FIFO	mkfifo() + open()	write()	read()	close()	unlink()
Message Queue	mq_open()	mq_send()	mq_receive()	mq_close()	mq_unlink()
Shared Memory	shm_open()	Memory Write	Memory Read	close()	shm_unlink()
Semaphore	sem_open() / sem_init() ()	sem_post()	sem_wait()	sem_close()	sem_unlink()
Socket	socket()	send()	recv()	close()	Auto

API Parameter Reference

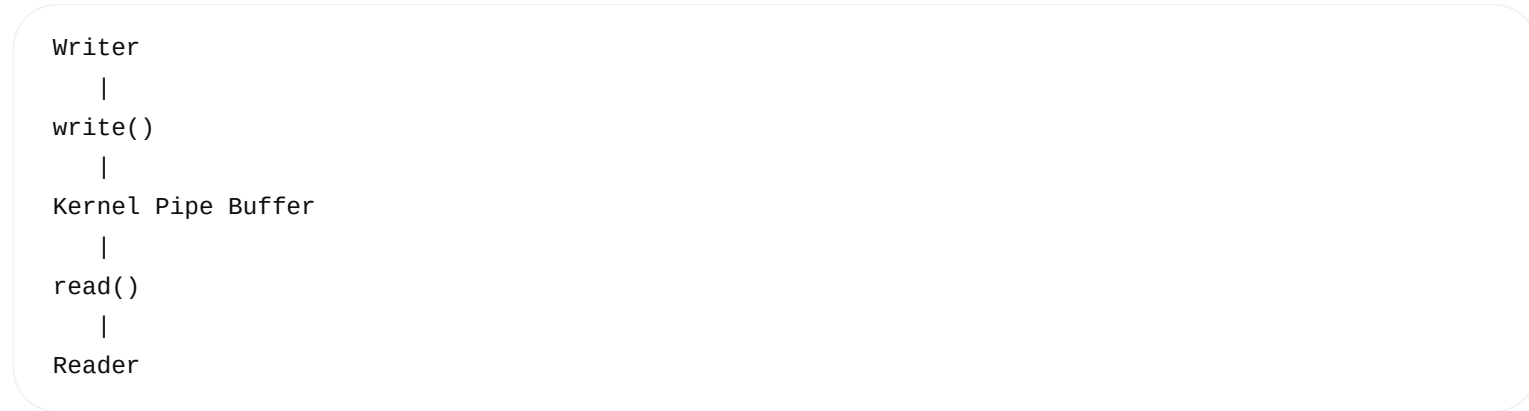
1. pipe()

```
int pipe(int fd[2]);
```

Parameter	Description
fd[0]	Read End

Parameter	Description
fd[1]	Write End

Flow



2. mkfifo()

```
int mkfifo(
    const char *pathname,
    mode_t mode);
```

Parameter	Description
pathname	FIFO file path
mode	Permission

Example

```
"/tmp/myfifo"
```

Visible

```
/tmp/myfifo
```

3. mq_open()

```
mqd_t mq_open(  
    const char *name,  
    int oflag,  
    mode_t mode,  
    struct mq_attr *attr);
```

Parameter	Description	Important Values
name	Queue Name	Must begin with "/"
oflag	Open Flags	O_CREAT, O_RDWR, O_RDONLY, O_NONBLOCK
mode	Permission	0666,0644
attr	Queue Properties	mq_attr

Queue Name

Correct

```
"/sensor_queue"
```

Wrong

```
"sensor_queue"
```

Visible

```
/dev/mqueue/sensor_queue
```

mq_attr

```
struct mq_attr
{
    long mq_flags;
    long mq_maxmsg;
    long mq_msgsize;
    long mq_curmsgs;
};
```

Field	Meaning
mq_flags	Blocking/Non Blocking
mq_maxmsg	Maximum Messages
mq_msgsize	Maximum Size of Message
mq_curmsgs	Current Queue Count

mq_send()

```
int mq_send(
    mqd_t mqdes,
    const char *msg_ptr,
    size_t msg_len,
    unsigned priority);
```

Parameter	Description
mqdes	Queue Descriptor
msg_ptr	Data
msg_len	Size
priority	Message Priority

mq_receive()

```
ssize_t mq_receive(  
    mqd_t mqdes,  
    char *msg_ptr,  
    size_t msg_len,  
    unsigned *priority);
```

Returns

Number of bytes received.

4. shm_open()

```
int shm_open(  
    const char *name,  
    int oflag,  
    mode_t mode);
```

Parameter	Description
name	Shared Memory Name
oflag	O_CREAT O_RDWR
mode	Permission

Correct

```
"/sensor_memory"
```

Wrong

```
"sensor_memory"
```

Wrong

```
"/tmp/sensor_memory"
```

Visible

```
/dev/shm/sensor_memory
```

ftruncate()

```
int ftruncate(  
    int fd,  
    off_t length);
```

Purpose

Resize Shared Memory.

mmap()

```
void *mmap(
    void *addr,
    size_t length,
    int prot,
    int flags,
    int fd,
    off_t offset);
```

Parameter	Description
addr	Usually nullptr
length	Mapping Size
prot	PROT_READ, PROT_WRITE
flags	MAP_SHARED
fd	Shared Memory FD
offset	Usually 0

sem_open()

```
sem_t* sem_open(  
    const char *name,  
    int oflag,  
    mode_t mode,  
    unsigned value);
```

Name

```
"/my_semaphore"
```

Visible

Usually under:

```
/dev/shm/
```

(depending on the implementation, semaphore objects often appear with a `sem.` prefix).

socket()

```
int socket(  
    int domain,  
    int type,  
    int protocol);
```

Example

```
socket(  
    AF_UNIX,  
    SOCK_STREAM,  
    0);
```

socketpair()

```
int socketpair(  
    int domain,  
    int type,  
    int protocol,  
    int sv[2]);
```

Used by

- WebKit
- Chromium
- Firefox

sendmsg()

```
ssize_t sendmsg(  
    int sockfd,  
    const struct msghdr *msg,  
    int flags);
```

Can send

- Normal Data
 - File Descriptor
 - Credentials
-

recvmsg()

```
ssize_t recvmsg(  
    int sockfd,  
    struct msghdr *msg,  
    int flags);
```

Receives

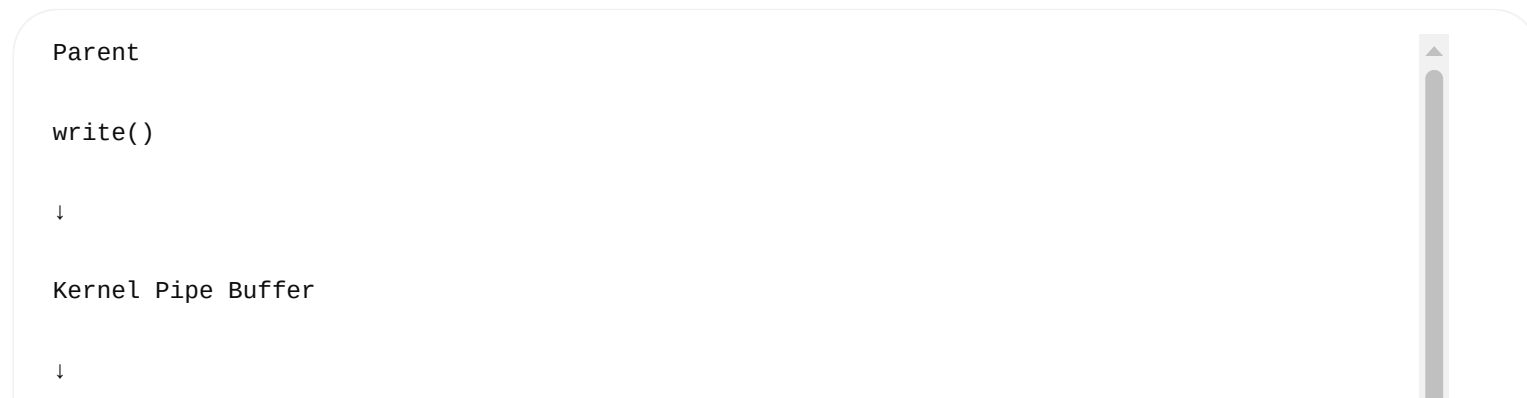
- Data
 - File Descriptor
-

File Visibility

IPC	Visible Location
Pipe	Not Visible
FIFO	<code>/tmp/myfifo</code>
POSIX Message Queue	<code>/dev/mqueue/</code>
Shared Memory	<code>/dev/shm/</code>
POSIX Semaphore	<code>/dev/shm/sem.*</code> (implementation-dependent)
UNIX Socket	Any filesystem path you bind it to (e.g. <code>/tmp/app.sock</code>) or abstract namespace on Linux
TCP Socket	Not File

Communication Flow

Pipe



read()

↓

Child

FIFO

Writer

↓

FIFO File

↓

Kernel

↓

Reader

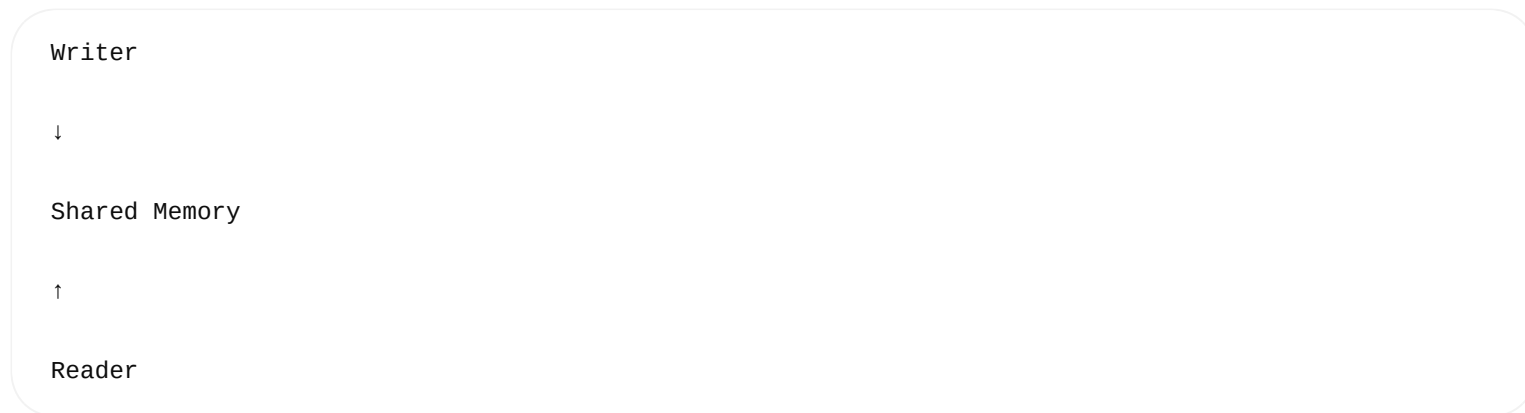
Message Queue

Sender

↓



Shared Memory



No kernel copy after mapping.

Shared Memory + Semaphore

Writer

↓

`sem_wait()`

↓

Write Memory

↓

`sem_post()`

↓

Reader

↓

`sem_wait()`

↓

Read

↓

`sem_post()`

UNIX Socket

Client

↓

send()

↓

Kernel Socket Buffer

↓

recv()

↓

Server

WebKit IPC

UI Process

↓

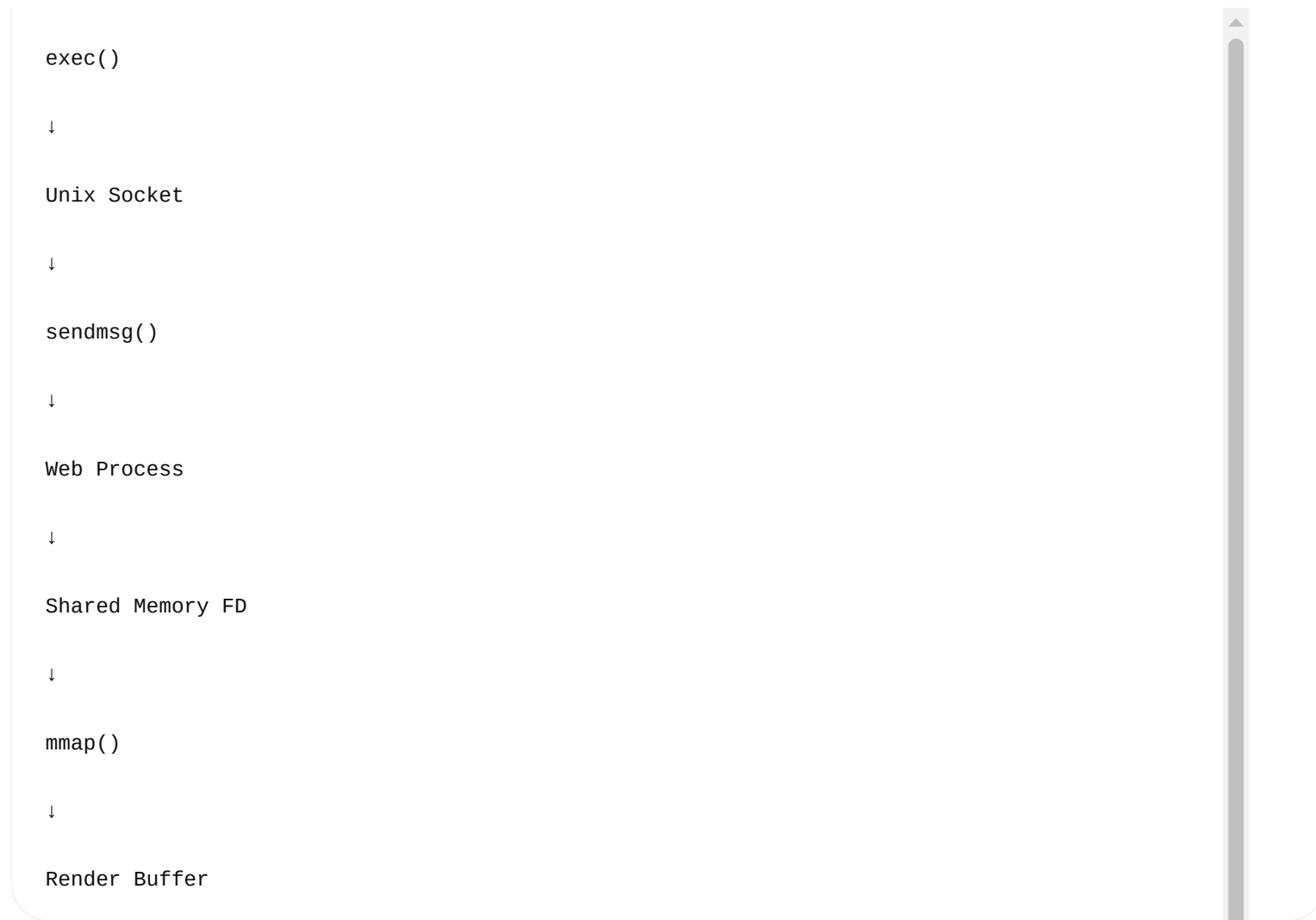
socketpair()

↓

fork()

↓





User Space vs Kernel Space

IPC	User Space	Kernel Space
Pipe	Read/Write Buffers	Pipe Buffer

IPC	User Space	Kernel Space
FIFO	Read/Write Buffers	FIFO Buffer
Message Queue	User Message	Queue Storage
Shared Memory	Same Memory Mapping	Memory Object Management
Socket	Send Buffer	Socket Buffer

Interview Points

Question	Answer
Why is Shared Memory fastest?	No data copy after mapping.
Why does Shared Memory need synchronization?	Multiple processes can access the same memory simultaneously, leading to race conditions.
Why are Message Queues slower?	Data is copied into and out of the kernel-managed queue.
Why use UNIX sockets in WebKit?	They support full-duplex communication and file descriptor passing (<code>SCM_RIGHTS</code>).
Why must <code>mq_open()</code> and <code>shm_open()</code> names start with <code>/</code> ?	POSIX specifies a namespace for named IPC objects. The leading <code>/</code> identifies the object within that namespace rather than as a filesystem path.

Question	Answer
Can <code>/tmp/test</code> be used as the name for <code>shm_open()</code> ?	No. Use <code>/test</code> . The implementation exposes it under <code>/dev/shm</code> , not <code>/tmp</code> .
Can <code>/tmp/test</code> be used for <code>mkfifo()</code> ?	Yes. FIFOs are actual filesystem objects, so you provide a real filesystem path.
Which IPC preserves message boundaries?	POSIX Message Queues.
Which IPC supports priorities?	POSIX Message Queues (<code>mq_send()</code> priority parameter).
Which IPC allows passing file descriptors?	UNIX domain sockets with <code>sendmsg()</code> / <code>recvmsg()</code> and <code>SCM_RIGHTS</code> .
Which IPC is used by WebKit for rendering buffers?	Shared memory for pixel buffers, Unix domain sockets for control messages and descriptor passing.

This table is a solid interview cheat sheet for **Embedded Linux, Linux System Programming, IPC,** and **WebKit/Chromium** discussions.